

PHASE 4

Wallets, Ledgers & Immutable Transactions

Secure Living — Financial Architecture Specification

1. Purpose

Phase 4 establishes the financial backbone of Secure Living. It converts existing wallet, transaction, escrow, rent, receipt, and deposit features into a strict trust-accounting layer where every shilling is traceable, every balance is derived, and posted financial records cannot be edited.

2. Core Outcome

A reliable double-entry ledger system covering user, property, and service wallets, plus rent, deposit, escrow, reserve, OPEX, and platform fee ledgers. All balances are calculated from posted ledger entries — never edited manually. Corrections happen only through reversal or adjustment entries with audit reasons.

3. Non-Negotiable Rules

- Every transaction links to its organisation, property, user, wallet, ledger, and reference record — no orphan entries.
- Transactions are append-only, posted only through a controlled service layer.
- All money-moving actions require idempotency keys to block duplicates.
- No application API may update or delete a posted transaction or ledger line.

4. Architecture Building Blocks

4.1 Wallets

Financial containers for users, properties, services, and the platform. Each wallet has an owner type, owner ID, organisation, currency, wallet type, freeze state, and status. Balance is a derived/cached value, never a source of truth.

4.2 Sub-Ledgers

Sit beneath wallets to identify the exact purpose of funds (e.g. rent, deposit, reserve, payout), keeping client funds cleanly segregated from platform money.

4.3 Transaction Journal

Records each financial event once, with full context: organisation, property, unit, lease, payer, payee, amount, type, status, idempotency key, and reason. Locked after posting.

4.4 Double-Entry Posting

Every transaction creates balanced debit and credit lines. Single-sided entries are disallowed except for approved, clearly marked migration opening balances.

4.5 Posting Service

The only path for creating ledger-backed transactions. It validates idempotency, wallet status, and permissions; creates the transaction, balanced ledger lines, and audit log; and rejects unbalanced or incomplete requests.

5. Required Ledger Types

Rent	Tenant rent receivable and payment tracking
Deposit	Tenant/landlord deposit and reserve obligations
Escrow	Trust-held funds pending conditions
Reserve	Landlord/company reserve balances
OPEX	Company operating funds and fees
Service	Repairs, inspections, professional work
Platform Fee	Management, escrow, coordination, listing fees
Payout / Refund / Adjustment / Clearing	Outbound, corrective, and in-transit movements

6. Controls & Safeguards

6.1 Reversals & Corrections

Mistakes are never fixed by editing or deleting. Reversal, adjustment, or refund entries link back to the original transaction, with authorising user and reason shown in the UI.

6.2 Idempotency

Duplicate requests (e.g. M-Pesa or bank callback retries) return the original response instead of creating new entries.

6.3 Freeze & Hold Readiness

Wallets and sub-ledgers carry flags/related hold records so Phase 5's escrow state machine can block payouts without breaking ledger integrity.

6.4 Opening Balances

Imported balances are controlled transactions with source file, migration batch ID, importer, approver, and timestamp — fully visible in ledger history.

6.5 Approvals

Reversals, fee overrides, opening balances, manual adjustments, and high-value payouts require role-based approval (Super Admin, Admin, Landlord, Tenant scopes).

6.6 Audit Logging

Every financial action creates an immutable audit entry: timestamp, user, role, action, old/new value, reason, authoriser, and linked resource.

7. APIs

Endpoints are organised under:

- /api/v1/wallets
- /api/v1/ledgers
- /api/v1/transactions
- /api/v1/accounting
- /api/v1/reconciliation
- /api/v1/fees
- /api/v1/finance-audit

Existing wallet/transaction routes are refactored to call the shared posting service rather than writing ledger entries directly.

8. Reconciliation

A reconciliation report compares ledger totals against expected balances, clearing accounts, and external references (bank/M-Pesa). At minimum: total trust balance = rent escrow + deposit escrow + reserves + clearing. Discrepancies trigger alerts and block related payouts until reviewed.

9. Role-Based Finance Visibility

Super Admin	Platform-wide trust totals, reconciliation status, reversal history, high-risk discrepancies
Admin	Organisation-scoped wallets, ledgers, approvals
Landlord	Own property wallets, rent/deposit ledgers, payouts, statements
Tenant	Own rent payments, receipts, deposit status, refunds
Provider	Job-related funds, approved payments, pending payouts

10. Deliverables

- Hardened wallet model (owner, organisation, type, currency, freeze, status)
- Sub-ledgers for rent, deposit, escrow, reserve, OPEX, service, fee, payout, refund, adjustment, clearing
- Append-only transaction journal with double-entry posting
- Derived balance API
- Reversal-only correction workflow
- Idempotency enforcement on all money-moving actions
- Opening balance import workflow
- Finance audit logs for sensitive actions
- Reconciliation report with discrepancy alerts
- Ledger-backed Banking, Transactions, Accounting, and Reports UI
- Role-scoped finance visibility for all user types
- Tests for double-entry, idempotency, reversal, scoping, and immutability

11. Implementation Order

Schema hardening → posting service → data migration → API updates → UI updates → reconciliation → statements → tests and acceptance checks. This sequencing keeps UI work from masking weak ledger logic.

12. Client Inputs Required

- Banking / Trust Account API
- M-Pesa Daraja API
- Payment Gateway Credentials (Stripe, PayPal, Airtel Money, T-Kash, etc.)
- SMS & Email Provider Credentials
- PDF Generation Service
- KYC / Identity Verification API
- Currency / FX Rate API
- Accounting Integrations (QuickBooks, Xero, etc.)
- Production Domains
- Webhook URLs & Secrets